

Clase 7

Estructuras lineales

Las listas, pilas y colas dependen de una estructura que se llama nodo. A su vez, el nodo tiene un dato de cualquier tipo y punteros a otros nodos.

En el caso de las listas, pueden tener punteros al siguiente y/o al anterior nodo, mientras que las pilas solo tienen al último y las colas al siguiente y al último.

Estructuras jerárquicas

Como los árboles, los cuales también cuentan con nodos, pero estos tienen un dato de cualquier tipo y dos punteros: uno al hijo de la izquierda y otro al hijo de la derecha.

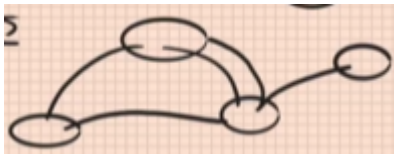
La cantidad de hijos depende del tipo de árbol. Por ejemplo, los binarios tienen solo 2 hijos.

La restricción que tienen los árboles y los diferencia de los grafos es que cada hijo puede heredar de un solo padre.

Grafos

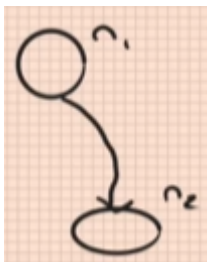
Son una generalización de los árboles que pierden esa restricción de un solo padre por hijo.

Por ejemplo:

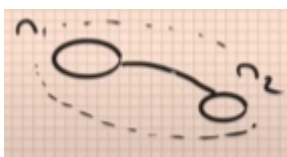


Los grafos pueden ser de dos grandes tipos:

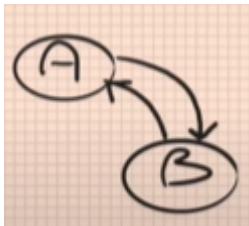
- Dirigidos → sólo se puede ir en un sentido.



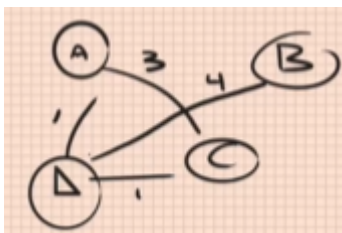
- No dirigidos → se puede ir en ambos sentidos. Son más flexibles para el desarrollo de algoritmos. Todas las uniones tienen que ser no dirigidas para que el grafo sea considerado no dirigido.



A pesar de que se representa con la misma arista, nosotros vamos a separarlas porque hay veces que no significa lo mismo en recursos y demás ir de un lado hacia al otro que viceversa.



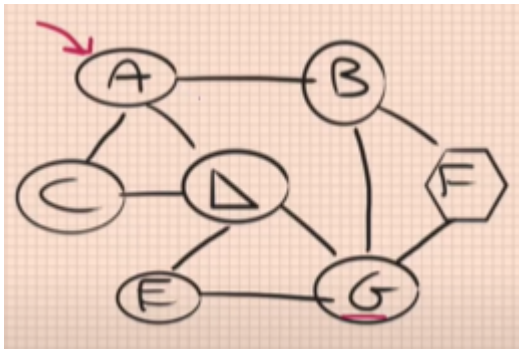
Tanto los grafos dirigidos como los no dirigidos, pueden ser ponderados. Esto refiere a la cantidad del recurso que consume cada arista. Como los tokens en las redes de Petri.



Métodos de búsqueda en grafos

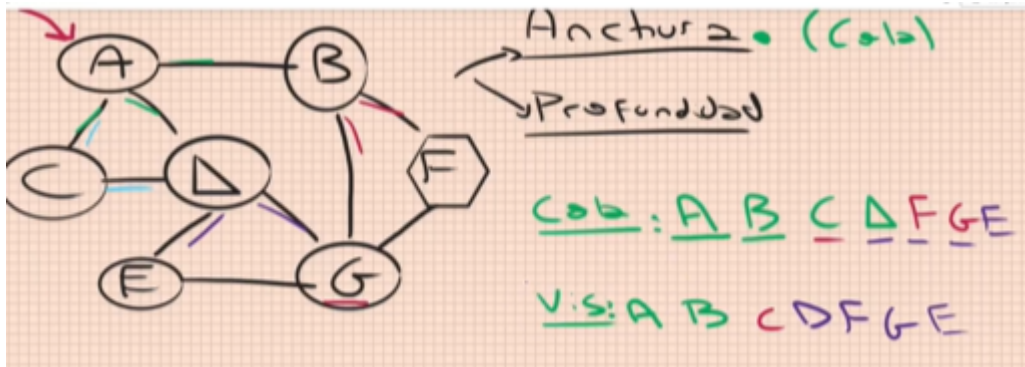
En primer lugar, debemos definir cuál es el nodo por el que queremos empezar y a cuál queremos llegar.

La cantidad de caminos viene dado por la cantidad máxima de aristas que hay elevado a la cantidad mínima de aristas que hay y no sea 1. En el caso del ejemplo serían 27 porque es 3^3 .

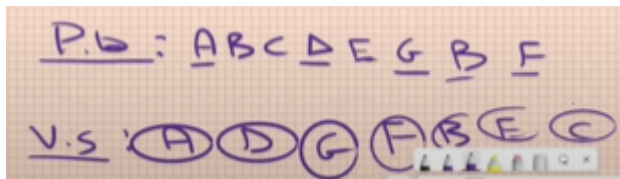


- Revisar a mano si es posible llegar desde el nodo inicial al buscado. Sin embargo, puede ocurrir esto de la multiplicidad de caminos.
- Recorrido en anchura
 - Tiene asociado una cola de los nodos en espera a recorrer.
 - También tenemos una de visitados para saber por cuáles ya pasamos.
 - Cuando visitamos un nodo, debemos continuar por alguno de sus adyacentes. Sin embargo, cuando hay más de uno hay que definir un criterio de desempate. Podemos tomar el orden del abecedario.

- Se encolan los nodos adyacentes a cada nodo visitado. Si ya está el nodo, no se encola.
- Se va desencolando hasta terminar la cola.
- Lo primero que se genera es el primer nivel de hijos del nodo inicial.

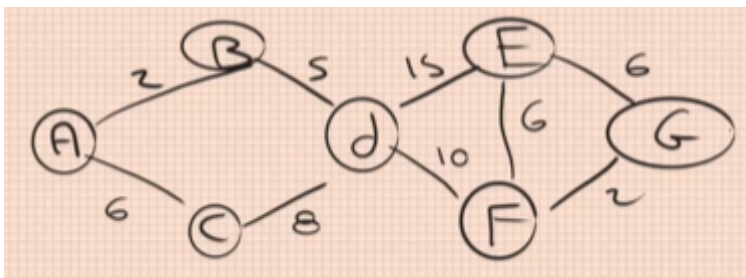


- Recorrido en profundidad
 - En vez de usar una cola, se usa una pila.
 - También tenemos los visitados/desapilados.
 - Debemos definir un criterio de desempate para los nodos adyacentes.
 - Cada vez que desapilamos un nodo debemos apilar sus adyacentes.
 - Lo primero que se genera es el camino más largo que hay en el grafo.



Tanto el recorrido por anchura como el por profundidad nos aseguran recorrer todos los nodos.

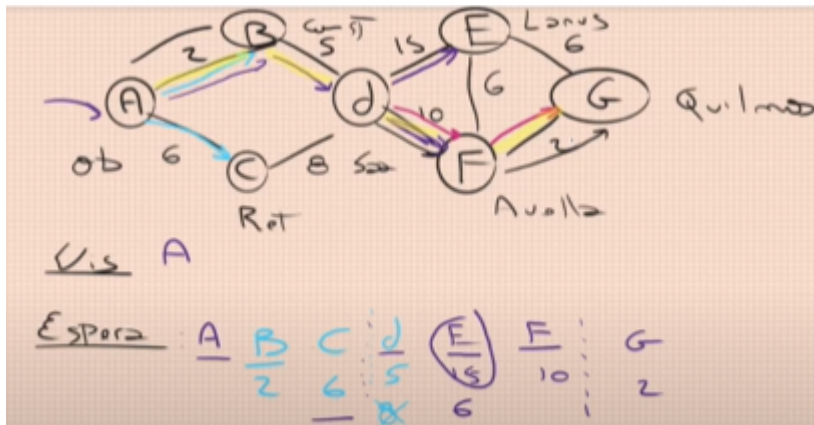
Por ejemplo:



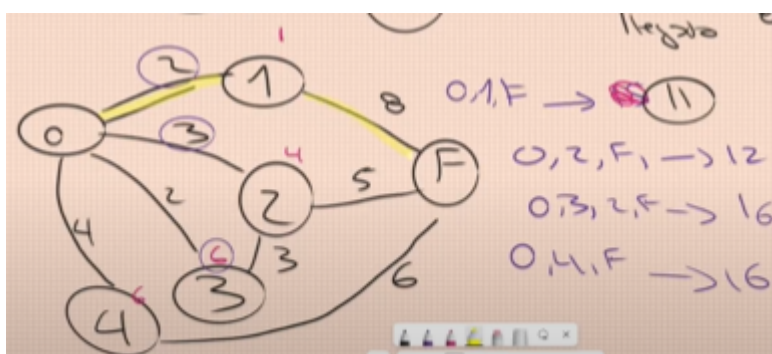
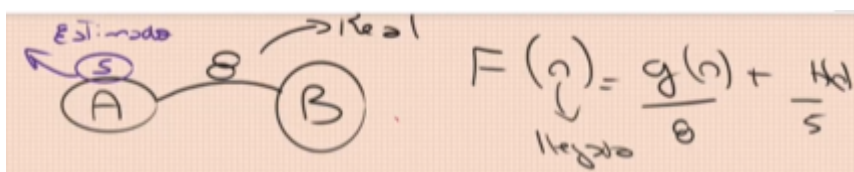
En los siguientes algoritmos importa la ponderación de los grafos:

- Dijkstra
 - Es para encontrar el mejor camino para llegar de un nodo a otro. Es decir, el más corto, más rápido, el que consume menos recursos.
 - Lista de visitados y otra para los que están en espera.
 - No solo se debe agregar a la lista de espera los adyacentes al nodo visitado, sino también su peso. Debemos visitar al de menor peso.

- Si a un nodo podemos llegar por más de un nodo, debemos quedarnos con la arista de menor peso, y ese valor indicarlo como el necesario para llegar.
- Si el nodo ya fue recorrido, por más que haya otra arista con un peso menor, no se actualiza.
- Consume mucho esfuerzo computacional porque hay bastantes idas y vueltas, pero sirve igual porque se calcula el camino una vez y lo guardamos.
- En el caso del ejemplo, el camino más corto es $A \rightarrow B \rightarrow D \rightarrow F \rightarrow G$.



- A* / AStar
 - Nos permite poner puntos de ruptura, lugares que permiten incorporar conocimiento previo. Esto es que los nodos, además del peso que tiene la arista que llega a ellos, tienen asociado el valor acumulado que costó llegar a ellos.
 - Sirve para el menos. Obtenemos el camino más corto pero utilizando información del pasado, es decir, cada nodo recuerda cuánto le costó realizar determinado camino.
 - Se suma el peso real de cada arista con el peso que estima el nodo.



La eficiencia de los algoritmos de grafos depende de la cantidad de vértices y de aristas.